



**IT134IU  
INTERNET OF THINGS LAB**

**Final Report:  
IoTs System with ESP8266/ESP32, Web Flask, Mit App Inventor  
and React Native**

**Student:      Phạm Tuấn Đăng Khoa**

**Student ID: ITITIU22087**

## I. Introduction

### 1.1 Background and Problem Statement

In the era of the Internet of Things (IoT), smart devices and connected systems are becoming ubiquitous, offering immense value for monitoring and control. However, building a complete IoT system from the ground up often involves complex integration between embedded hardware, cloud services, and diverse user interfaces. The challenge lies in creating an end-to-end solution that can reliably collect data from physical sensors and present it intuitively across multiple platforms, from a web dashboard to mobile applications.

### 1.2 Project Objectives

This project was undertaken to design and implement a comprehensive IoT system to address the aforementioned challenge. The specific objectives are as follows:

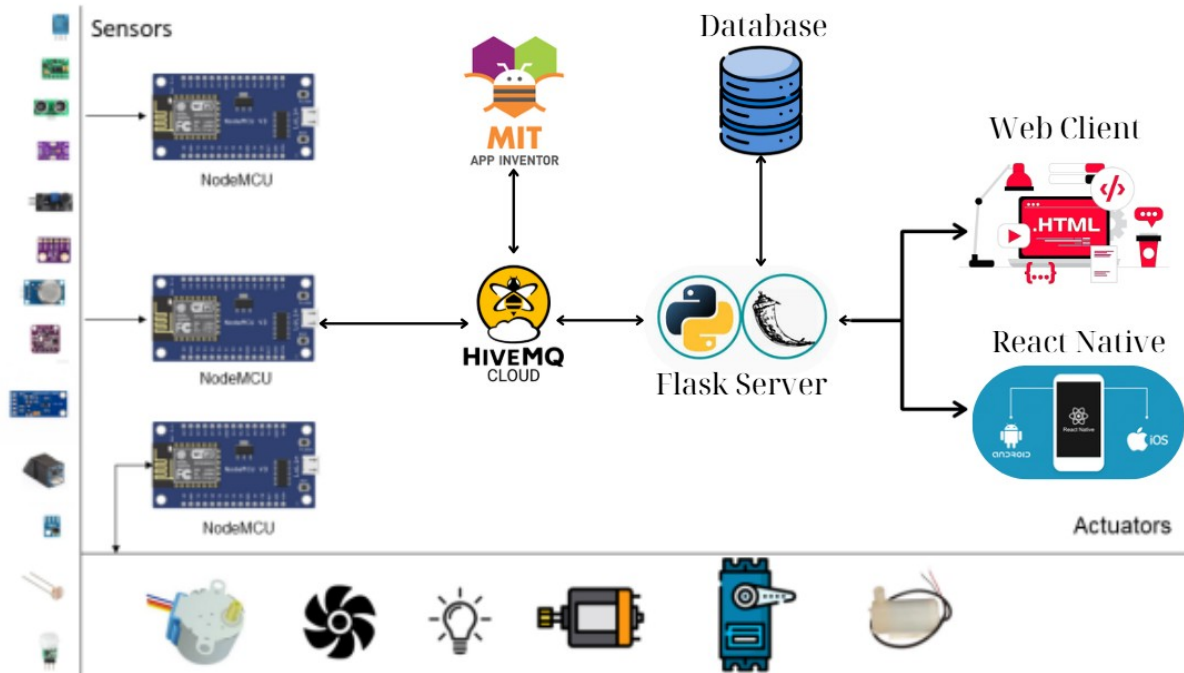
- **Hardware Design and Programming:** To utilize ESP8266/ESP32 modules for collecting data from various sensors (e.g., temperature, humidity, light) and transmitting this data to a server.
- **Backend Development:** To build a backend web application using the Flask framework in Python to receive, process, store data from the embedded devices in a database, and provide APIs for frontend applications.
- **Frontend Development:** To create user interfaces across multiple platforms:
  - ◆ **Web Interface:** Using HTML/CSS/JavaScript to display data on a web browser.
  - ◆ **Mobile Applications:** Developing apps on two different platforms to demonstrate versatility:
    - **MIT App Inventor:** To create a simple, intuitive Android app suitable for quick testing and educational purposes.
    - **React Native:** To develop a more professional cross-platform mobile application (iOS/Android) that connects to the backend API for a robust user experience.

### 1.3 Project Scope

The scope of this project encompasses the entire system architecture design, code development for the embedded hardware (ESP), the backend (Flask), and the various frontends (Web, MIT App Inventor, React Native). This report will detail the system architecture, database schema, API specifications, test plans, and the final results. The project's core focus is to demonstrate the successful integration of all specified technologies to create a complete and functional IoT solution.

## II. Analysis & Design

### 2.1 System Architecture



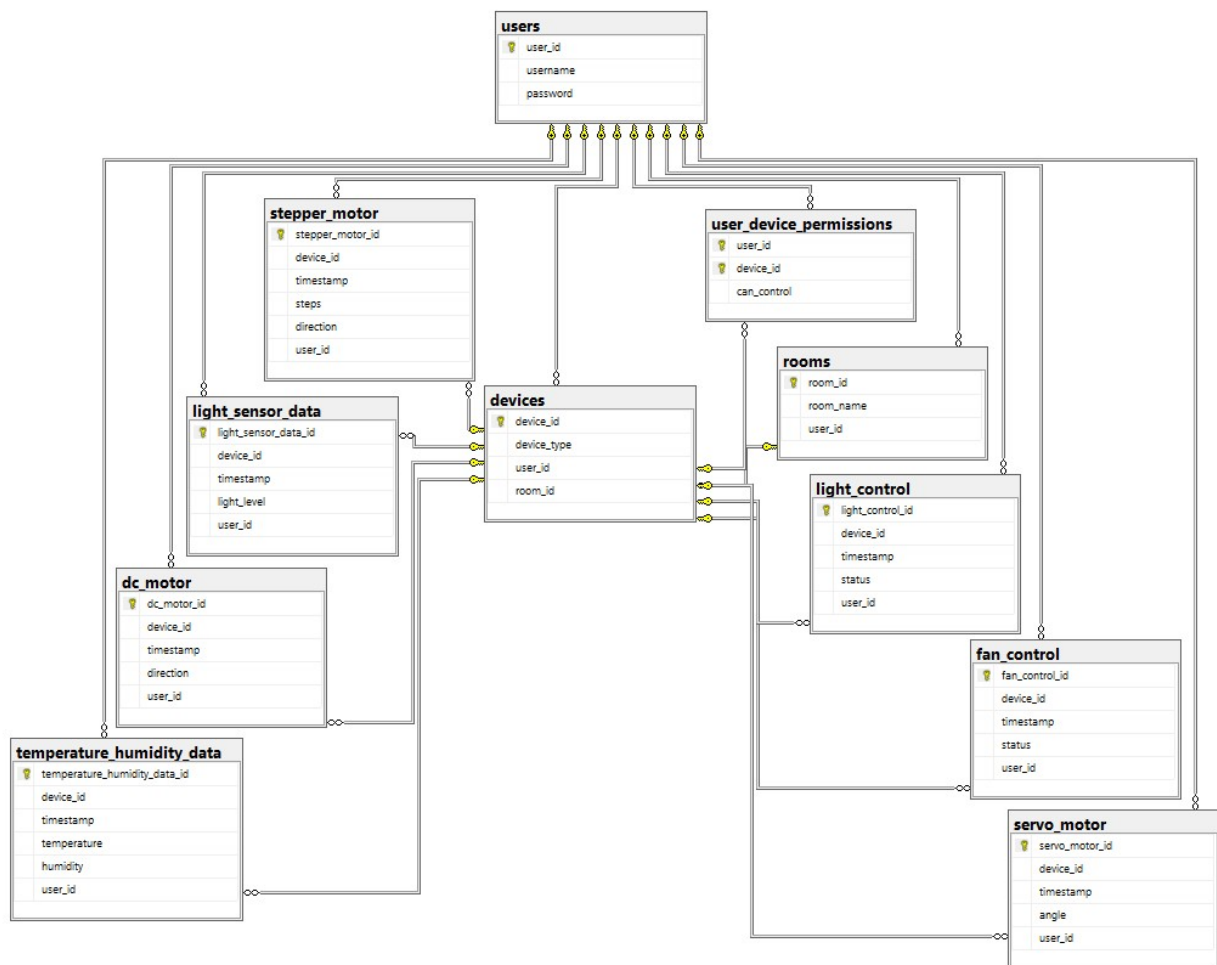
*Figure: Block Diagram*

The diagram illustrates the overall IoT system architecture, where all components communicate and exchange data through the MQTT Broker (HiveMQ Cloud) and the Flask Server.

- **IoT Devices (NodeMCU ESP8266/ESP32 + Sensors/Actuators)**
  - Each NodeMCU is connected to various sensors such as DHT11, light sensors, PIR motion sensors, etc., to collect environmental data.
  - It also controls actuators such as stepper motors, fans, lights, servo motors, and relays.
  - Sensor data is sent to HiveMQ Cloud using the MQTT protocol, and actuator commands are received from the system via MQTT.
- **MQTT Broker – HiveMQ Cloud**
  - Acts as a central hub for exchanging data between NodeMCUs, the Flask Server, and user applications.
  - NodeMCUs publish sensor data and subscribe to receive control commands.
  - The Flask Server also subscribes to receive data and publishes control commands back to the devices.
- **Flask Server (Backend)**
  - Processes data from the MQTT Broker and provides APIs for frontend applications (Web Client, React Native, MIT App Inventor).
  - When sensor data is received from MQTT, it is stored in the Database.
  - When a control request is received from a frontend, the Flask Server sends a corresponding command via MQTT Broker to the IoT devices.
- **Database**
  - Stores sensor readings, device operation history, and user information.
  - The Flask Server retrieves data from the database to display to users or for further analysis.

- **Frontend**
  - Web Client: Displays sensor data and allows device control through a browser interface.
  - React Native App: Cross-platform mobile app for remote monitoring and control.
  - MIT App Inventor App: Quickly developed mobile app for basic device control.
  - All frontends interact with the Flask Server via REST APIs.
- **Data Flow Summary:**
  - Sensor data flow: Sensors → NodeMCU → MQTT Broker → Flask Server → Database → Frontend.
  - Control flow: Frontend → Flask Server → MQTT Broker → NodeMCU → Actuators.

## 2.2 Database Schema



*Figure: Database Diagrams*

### Table Descriptions

#### 1. users Table

This table stores basic information about the users.

- user\_id: Primary key, auto-incrementing, uniquely identifies a user.
- username: The user's login name, not null, and must be unique.
- password: The user's hashed password, not null.

#### 2. rooms Table

This table manages the rooms that users create to organize their devices.

- room\_id: Primary key, auto-incrementing, uniquely identifies a room.
- room\_name: The name of the room, not null.

- `user_id`: Foreign key linked to the users table, indicating the user who owns the room.

### **3. devices Table**

This is the main table for storing information about all IoT devices in the system.

- `device_id`: Primary key, auto-incrementing, uniquely identifies a device.
- `device_type`: The type of device (e.g., `light_sensor`, `dc_motor`), not null.
- `user_id`: Foreign key linked to the users table, indicating the user who owns the device.
- `room_id`: Foreign key linked to the rooms table, indicating the room where the device is located.

### **4. dc\_motor Table**

This table records the control commands sent to a DC motor.

- `dc_motor_id`: Primary key, auto-incrementing, uniquely identifies a record.
- `device_id`: Foreign key linked to the devices table, indicating the DC motor device being controlled.
- `[timestamp]`: The time when the command was recorded.
- `direction`: The rotation direction of the motor (forward or reverse), not null.
- `user_id`: Foreign key linked to the users table, indicating the user who executed the command.

### **5. fan\_control Table**

This table stores the operational status of a fan.

- `fan_control_id`: Primary key, auto-incrementing, uniquely identifies a record.
- `device_id`: Foreign key linked to the devices table, indicating the fan device being controlled.
- `[timestamp]`: The time when the command was recorded.
- `status`: The status of the fan (1 for ON, 0 for OFF), not null.
- `user_id`: Foreign key linked to the users table, indicating the user who executed the command.

### **6. light\_control Table**

This table records the control commands sent to a light.

- `light_control_id`: Primary key, auto-incrementing, uniquely identifies a record.
- `device_id`: Foreign key linked to the devices table, indicating the light device being controlled.
- `[timestamp]`: The time when the command was recorded.
- `status`: The status of the light (1 for ON, 0 for OFF), not null.
- `user_id`: Foreign key linked to the users table, indicating the user who executed the command.

### **7. light\_sensor\_data Table**

This table stores data from a light sensor.

- `light_sensor_data_id`: Primary key, auto-incrementing, uniquely identifies a record.
- `device_id`: Foreign key linked to the devices table, indicating the light sensor device.
- `[timestamp]`: The time the data was recorded.
- `light_level`: The measured light level, not null.
- `user_id`: Foreign key linked to the users table, indicating the user who owns this device.

### **8. servo\_motor Table**

This table records the control commands sent to a servo motor.

- servo\_motor\_id: Primary key, auto-incrementing, uniquely identifies a record.
- device\_id: Foreign key linked to the devices table, indicating the servo device being controlled.
- [timestamp]: The time when the command was recorded.
- angle: The rotation angle of the servo motor, not null.
- user\_id: Foreign key linked to the users table, indicating the user who executed the command.

### **9. stepper\_motor Table**

This table records the control commands sent to a stepper motor.

- stepper\_motor\_id: Primary key, auto-incrementing, uniquely identifies a record.
- device\_id: Foreign key linked to the devices table, indicating the stepper motor device being controlled.
- [timestamp]: The time when the command was recorded.
- steps: The number of steps to move, not null.
- direction: The rotation direction (clockwise or counter-clockwise), not null.
- user\_id: Foreign key linked to the users table, indicating the user who executed the command.

### **10. temperature\_humidity\_data Table**

This table stores data from a temperature and humidity sensor.

- temperature\_humidity\_data\_id: Primary key, auto-incrementing, uniquely identifies a record.
- device\_id: Foreign key linked to the devices table, indicating the sensor device.
- [timestamp]: The time the data was recorded.
- temperature: The measured temperature value, not null.
- humidity: The measured humidity value, not null.
- user\_id: Foreign key linked to the users table, indicating the user who owns this device.

### **11. user\_device\_permissions Table**

This table manages access and control permissions for users over specific devices.

- user\_id: Primary key and foreign key linked to the users table.
- device\_id: Primary key and foreign key linked to the devices table.
- can\_control: A BIT type column with a default value of 1, indicating whether the user has control permissions over the device.
- The combination of (user\_id, device\_id) forms the composite primary key to ensure a single permission record per user-device pair.

## **2.3 API Specifications**

### **2.3.1 Authentication and Admin APIs**

#### **Endpoint: /register**

- Method: POST
- Description: Creates a new user account.
- Request Body (Form Data): username, password
- Response: Redirects to the login page.

#### **Endpoint: /login**

- Method: POST

- Description: Authenticates a user and establishes a session.
- Request Body (Form Data): username, password
- Response: Redirects to the homepage (/) or the admin page (/admin/client) if the user has admin credentials.

**Endpoint: /logout**

- Method: GET
- Description: Ends the current user session and logs the user out.
- Response: Redirects to the login page.

**Endpoint: /admin/client**

- Method: GET
- Description: Renders the admin dashboard for managing user permissions and devices.

**Endpoint: /admin/toggle\_permission**

- Method: POST
- Description: Toggles a specific user's control permission for a device.
- Request Body (Form Data): user\_id, device\_id
- Response: Redirects back to the admin page.

**Endpoint: /admin/assign\_device**

- Method: POST
- Description: Assigns a new device to a specific user and room, with default control permissions.
- Request Body (Form Data): user\_id, room\_id, device\_type
- Response: Redirects back to the admin page.

### 2.3.2 Device and Room Management APIs

**Endpoint: /get\_rooms**

- Method: GET
- Description: Retrieves a list of rooms owned by the logged-in user.
- Response (JSON): [{ 'id': 1, 'name': 'Living Room' }, ...]

**Endpoint: /add\_room**

- Method: POST
- Description: Adds a new room for the current user.
- Request Body (JSON): { 'room\_name': 'Room Name' }
- Response (JSON): { 'success': True, 'room\_id': 1, 'room\_name': 'Room Name' }

**Endpoint: /get\_devices/<int:room\_id>**

- Method: GET
- Description: Fetches a list of devices within a specified room, including their control permissions.
- Response (JSON): [{ 'id': 101, 'type': 'light\_control', 'room\_id': 1, 'can\_control': True }, ...]

**Endpoint: /add\_device**

- Method: POST
- Description: Adds a new device to a specific room for the current user.
- Request Body (JSON): { 'room\_id': 1, 'device\_type': 'light\_control' }

- Response (JSON): {'success': True, 'device\_id': 101, 'device\_type': 'light\_control'}

**Endpoint: /api/devices/<int:device\_id>**

- Method: DELETE
- Description: Deletes a device and all its associated data.
- Response (JSON): {'success': True, 'message': 'Device deleted successfully'}

**Endpoint: /api/rooms/<int:room\_id>**

- Method: DELETE
- Description: Deletes a room and all associated devices and data.
- Response (JSON): {'success': True, 'message': 'Room and all devices have been successfully deleted'}

### 2.3.3 Sensor Data APIs

**Endpoint: /sensor\_data/<int:room\_id>**

- Method: GET
- Description: Retrieves the most recent sensor data (temperature, humidity, light, PIR) for a given room.
- Response (JSON): {'temperature': 25.5, 'humidity': 70.2, 'light': 500, 'pir': 'Detected'}

**Endpoint: /data\_visualization**

- Method: GET
- Description: Fetches the latest 100 sensor data records for visualization purposes.
- Response (JSON): [['2025-08-16T09:00:00', 25.5, 70.2, 500], ...]

### 2.3.4 Actuator Control APIs

**Endpoint: /control**

- Method: POST
- Description: Controls a relay (light) based on the provided action.
- Request Body (JSON): {'device\_id': 101, 'action': 'relay\_on', 'room\_name': 'Living Room'}
- Response (JSON): {'success': True}

**Endpoint: /control\_fan**

- Method: POST
- Description: Controls the fan's status.
- Request Body (JSON): {'device\_id': 102, 'action': 'on', 'room\_name': 'Living Room'}
- Response (JSON): {'success': True}

**Endpoint: /control\_stepper**

- Method: POST
- Description: Controls the stepper motor's direction.
- Request Body (JSON): {'device\_id': 103, 'action': 'cw', 'room\_name': 'Living Room'}
- Response (JSON): {'success': True}

**Endpoint: /control\_motor**

- Method: POST
- Description: Controls the DC motor's direction and status.
- Request Body (JSON): {'device\_id': 104, 'action': 'forward', 'room\_name': 'Living Room'}
- Response (JSON): {'success': True}

**Endpoint: /control\_servo**

- Method: POST
- Description: Sets the angle of the servo motor.
- Request Body (JSON): {'device\_id': 105, 'angle': 90, 'room\_name': 'Living Room'}
- Response (JSON): {'success': True}

**Endpoint: /actuator\_status**

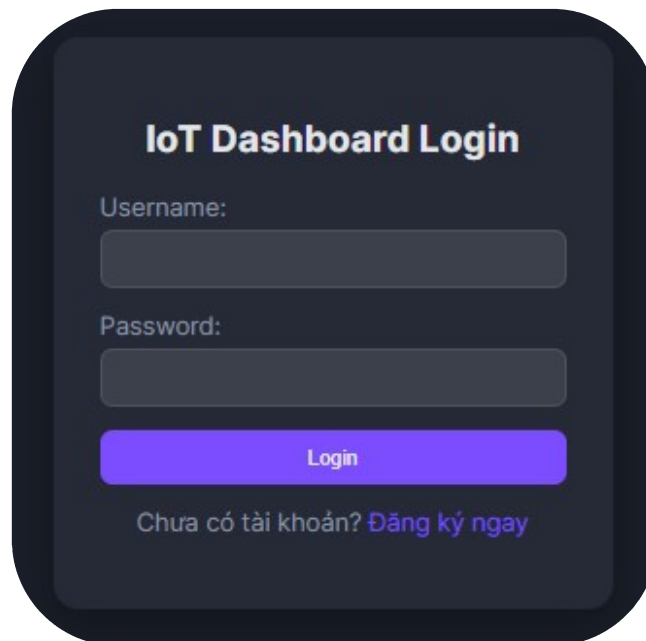
- Method: GET
- Description: Retrieves the latest status of all actuators and the PIR sensor from the backend's memory cache and database.
- Response (JSON): {'light': 'ON', 'fan': 'OFF', 'motor': 'STOP', 'stepper': 'CW', 'servo': 90, 'pir': 'No Motion'}

## 2.4 GUI Mockups

### 2.4.1 Web Interface Design

The web interface is designed as a centralized dashboard for real-time monitoring and device management. The layout is clean and intuitive, focusing on data visualization.

**Login Page:**



*Figure: Login Page*

- Description: The user interface begins with a simple and clean login page.
- Key Components: It includes two main input fields for Username and Password, along with a Login button. Below the login button is a link to the registration page for new users. This design ensures easy access to the system.

## Dashboard Page:

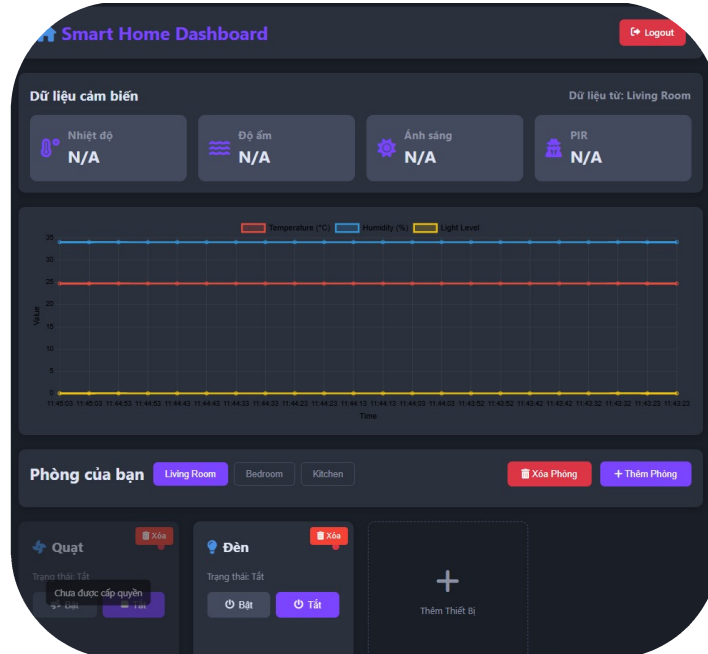


Figure: Dashboard Page

- Description: After logging in, the user is redirected to the main dashboard, which displays all the data and control options.
- Key Components:
  - ◆ **Sensor Data Section**: This section shows important readings from sensors, such as Temperature, Humidity, Light Level, and motion detection from a PIR sensor. The data is presented in separate blocks, allowing users to easily monitor the current status.

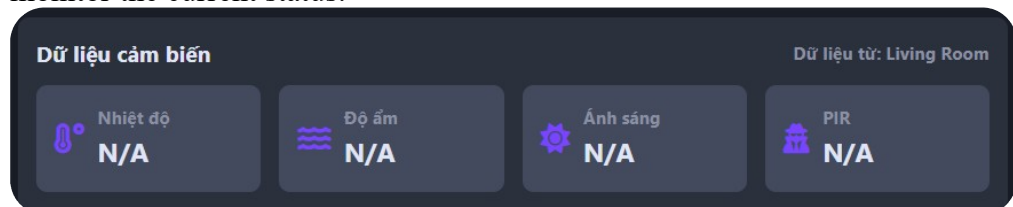


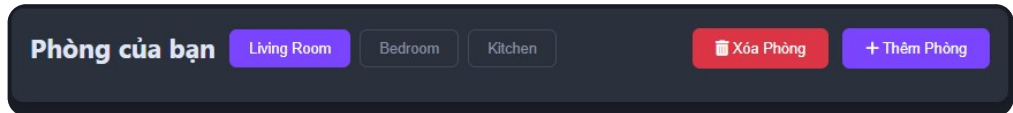
Figure: Sensor Data Section

- ◆ **Data Chart**: A line chart displays historical data for temperature, humidity, and light over time, helping users track environmental trends within the home.



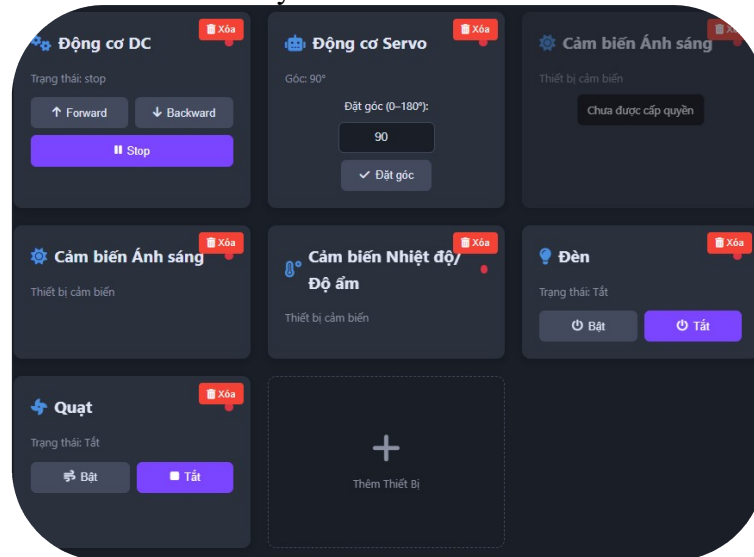
Figure: Data Chart

- ◆ **Room & Device Management Section**: This part allows users to select different rooms to view their corresponding data and devices. Users can also add new rooms with the "+ Add Room" and "Delete Room" buttons.



*Figure: Room & Device Management Section*

- ◆ **Device Control Section:** This section shows the actuators in the selected room (e.g., Fan, Light). Users can see the current status of each device (On/Off) and use corresponding buttons to control them. A "Delete" button is also available to remove devices from the system.



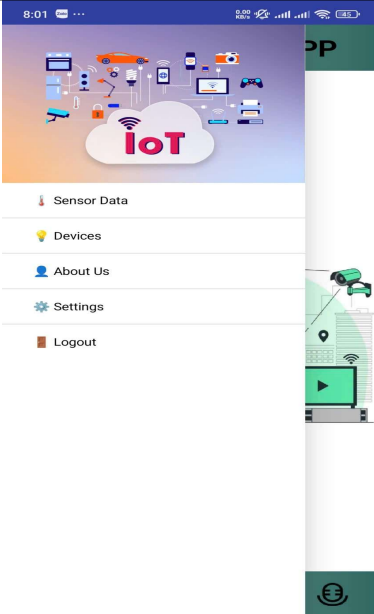
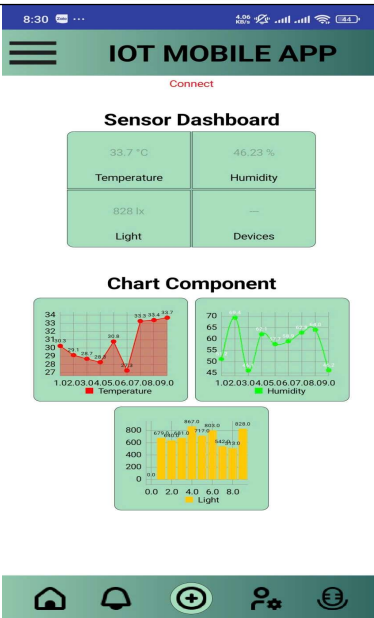
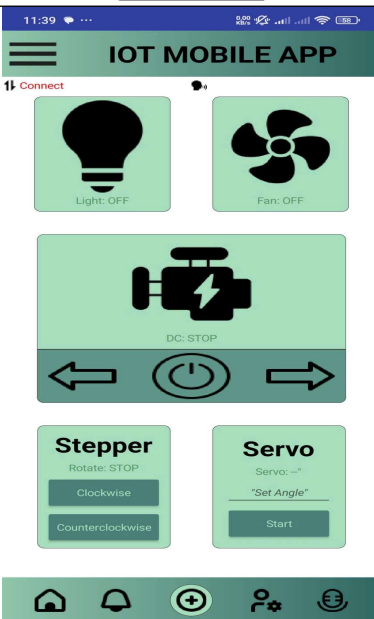
*Figure: Device Control Section*

### 2.4.2 MIT App Inventor Interface Design

The interface on MIT App Inventor was developed primarily for educational purposes and to demonstrate the connection between hardware, the MQTT server, and the mobile application. Due to the use of a low-code (drag-and-drop) programming platform, this application focuses on core functions such as direct control of actuators and displaying real-time sensor data. Although it lacks full-fledged features like user management or complex data storage found in the web interface, this application plays a crucial role in proving the feasibility of the operational model.

*Table: MIT App Inventor Interface Design*

| Figure                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Description                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p>The image shows the main screen of the "IOT MOBILE APP". At the top, there's a status bar with the time 8:01 and various icons. Below it is a green header with a hamburger menu icon and the text "IOT MOBILE APP". The main area features a large, stylized illustration of a person sitting on a green beanbag chair, holding a smartphone. In the background, there are city buildings, a large "IoT" text, a drone, and a laptop. At the bottom, there's a green navigation bar with icons for home, notifications, a plus sign (add), settings, and a profile.</p> | <p><b>Main Screen:</b> The app opens with a clear header, "IOT MOBILE APP". A simple illustration related to IoT is displayed, setting a welcoming tone.</p> |

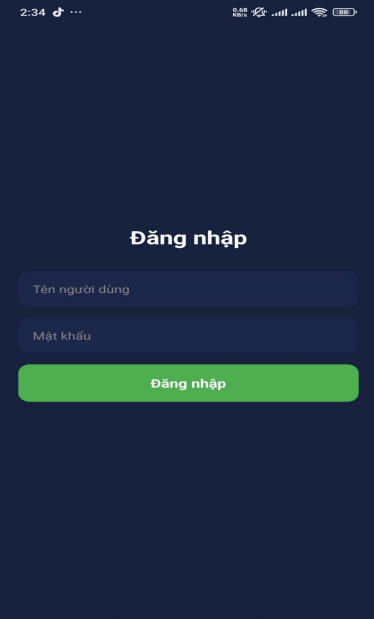
|                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p><b>Navigation:</b> A hamburger menu icon in the top-left corner opens a sidebar for navigation. This menu provides quick access to key sections:</p> <ul style="list-style-type: none"> <li>➤ Sensor Data</li> <li>➤ Devices</li> <li>➤ About Us</li> <li>➤ Settings</li> <li>➤ Logout</li> </ul>                                                                                                                                                                                                                                                                                                        |
|   | <p><b>Sensor Dashboard:</b> This section presents real-time sensor data in a grid layout. Each block displays a sensor's type and its current reading, such as Temperature, Humidity, and Light.</p> <p><b>Data Charts:</b> Below the sensor data, the app shows line and bar charts to visualize historical sensor data for temperature, humidity, and light. This feature allows users to easily track trends over time.</p>                                                                                                                                                                              |
|  | <p><b>Device Control:</b> The control screen provides a series of large, icon-based buttons for direct interaction with actuators.</p> <ul style="list-style-type: none"> <li>➤ Devices like the Light and Fan have blocks showing their status (e.g., "ON" or "OFF").</li> <li>➤ The DC Motor control block includes buttons for directional movement ("forward," "backward") and a stop button.</li> <li>➤ Separate blocks are dedicated to the Stepper motor (with options for "Clockwise" and "Counterclockwise" rotation) and the Servo motor (with a "Start" button to control its angle).</li> </ul> |

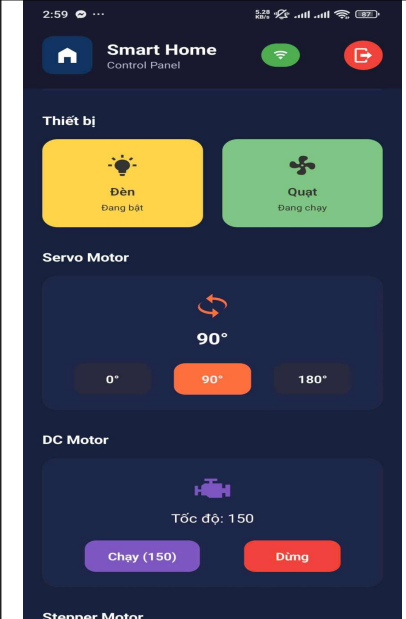
### 2.4.3 React Native Interface Design

The mobile application interface built with React Native aims to provide a modern and professional user experience on both Android and iOS. While designed with a sleek interface and the potential to be a complete product, the current version is still in its initial development phase.

- **Current Status and Purpose:** This application's primary focus is to demonstrate the ability to communicate with the Flask Server backend via API requests. However, it does not yet integrate all the complex functionalities like room and device management. Instead, it still uses the old MQTT topics and messages for device control, similar to the MIT App Inventor application. This makes the React Native app proof that the operational model can be extended to other mobile platforms.

*Table: React Native Interface Design*

| Figure                                                                              | Description                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | <p><b>Login Screen:</b> The app starts with a login screen featuring input fields for Username and Password and a Login button. The design is simple and visually appealing, consistent with modern mobile apps.</p>                                                                                                                     |
|  | <p><b>Dashboard:</b> The main dashboard is designed to be intuitive and informative. It displays real-time sensor data for temperature, humidity, and light level in distinct, card-based sections. Below the live data, a line chart visualizes the historical trend of sensor readings, allowing users to track changes over time.</p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p>The image shows a mobile app interface titled 'Smart Home Control Panel'. It features a dark blue background with several control cards. At the top, there's a status bar showing the time 2:59 and various connectivity icons. Below the title, there are two main cards: a yellow one for 'Đèn' (Light) with a lightbulb icon and a green one for 'Quạt' (Fan) with a fan icon. Both have 'Đang bật' (On) status. Below these are three motor control sections: 'Servo Motor' with a 90° angle display and buttons for 0°, 90°, and 180°; 'DC Motor' with a speed display of 150 and buttons for 'Chạy (150)' (Run) and 'Dừng' (Stop); and 'Stepper Motor' at the bottom.</p> | <p><b>Device Control:</b> A separate control panel allows for direct interaction with various actuators. This screen is organized into dedicated sections for each device type:</p> <ul style="list-style-type: none"> <li>➤ <b>Light and Fan:</b> Toggles display their current status (e.g., "Đang bật" - "On," "Đang chạy" - "Running").</li> <li>➤ <b>Servo Motor:</b> A numerical display shows the current angle, with buttons to quickly set the angle to 0°, 90°, or 180°.</li> <li>➤ <b>DC Motor:</b> Provides buttons to control the motor's direction and status, with options like "Chạy" (Run) and "Dừng" (Stop).</li> <li>➤ <b>Stepper Motor:</b> Offers buttons for directional control (e.g., "Clockwise" and "Counterclockwise").</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### III. Code Description

#### 3.1 ESP Module (Arduino/ESP)

##### 3.1.1 Configuration & Connection

The code uses libraries like WiFi.h and PubSubClient.h to establish a secure wireless connection. Key information like Wi-Fi and MQTT login credentials, as well as physical pin assignments on the board, are clearly defined. The setup() function initializes these pins and establishes the Wi-Fi and MQTT connections.

##### 3.1.2 Data Collection & Publishing

This functionality is handled in the main loop() function. The code reads values from sensors (DHT11, light, PIR) and sends them to the MQTT broker on a 10-second cycle. The following code snippet illustrates how the data is collected and published to MQTT:

```
// Send sensor data every 10s

static unsigned long lastSensorTime = 0;

if (millis() - lastSensorTime >= 10000) {

    lastSensorTime = millis();

    float humidity = dht.readHumidity();

    float temperature = dht.readTemperature();

    int lightLevel = analogRead(lightPin);
```

```
if (!isnan(humidity) && !isnan(temperature)) {  
    String payload = String(humidity) + "," + String(temperature) + "," + String(lightLevel);  
    client.publish("home/khoa/Bedroom/sensors", payload.c_str());  
}  
}
```

### 3.1.3 Command Reception & Device Control

This is the core of the remote control functionality, handled by the `mqttCallback()` function. This function listens for messages from control topics. When a message is received, it parses the content to perform the corresponding action.

```
void mqttCallback(char* topic, byte* message, unsigned int length) {  
    String msg;  
    for (unsigned int i = 0; i < length; i++) {  
        msg += (char)message[i];  
    }  
    Serial.print("Message received: ");  
    Serial.println(msg);  
  
    if (msg == "relay_on") {  
        digitalWrite(relayPin, HIGH);  
    } else if (msg == "fan_on") {  
        digitalWrite(fanRelayPin, HIGH);  
    } else if (msg.startsWith("servo:")) {  
        controlServo(msg.substring(6));  
    }  
    // ... and other control commands  
}
```

### 3.1.4 Device Control Functions

The code includes separate functions to control each type of actuator, such as the DC motor, stepper motor, and servo. These functions are called from `mqttCallback()` to execute specific control commands.

### 3.1.5 Additional Features

The `reconnectMQTT()` function ensures the device automatically reconnects to the server if the connection is lost, helping the system maintain stable and continuous operation.

## 3.2 Flask Backend (app.py)

### 3.2.1 Connectivity

The code establishes the necessary connections for the system to function.

- **SQLite Database:** It uses the `sqlite3` library to connect to and manage the database. This database stores information about users, rooms, devices, and sensor data.
- **MQTT Broker:** It uses the `paho.mqtt.client` library to connect to the HiveMQ Cloud MQTT broker. Once connected, it automatically subscribes to sensor data and control topics for the specified user, ensuring communication with the ESP devices.

```
# Kết nối MQTT và bắt đầu vòng lặp
```

```
try:
```

```
    mqttClient.connect(mqttBroker, mqttPort)
```

```
# Đăng ký lắng nghe các topic cụ thể cho người dùng đã được cung cấp
```

```
mqttClient.subscribe(f"home/{username}/+/sensors")
```

```
mqttClient.subscribe(f"home/{username}/+/control")
```

```
mqttClient.subscribe(f"home/{username}/+/pir")
```

```
mqttClient.loop_start()
```

```
print("✓ MQTT client đã khởi động.")
```

```
except Exception as e:
```

```
    print(f"✗ Lỗi khi kết nối MQTT: {e}")
```

### 3.2.2 RESTful APIs & Data Processing

This module provides RESTful APIs for the web and mobile user interfaces to interact with the backend.

- Sensor and Control APIs: Endpoints such as `/sensor_data`, `/control`, and `/control_fan` allow the frontends to retrieve the latest sensor data and send control commands to devices via MQTT.
- MQTT Data Processing: The `on_message` function is crucial for listening to messages from the ESP devices. When sensor data is received, it parses the content and stores it in the database. It also updates the status of actuators in a cache for quick responses to the frontends.

### 3.2.3 Authentication and Management

The Flask backend is responsible for managing users and access permissions.

- User Authentication: It handles user registration (`/register`), login (`/login`), and logout (`/logout`) requests by interacting with the database to verify user information and manage the session.
- Admin Features: It provides dedicated endpoints for administrators, allowing them to view user information, toggle device control permissions (`/admin/toggle_permission`), and add new devices to the system (`/admin/assign_device`).

## **IV. Challenges & Solutions**

### **Challenges**

The project encountered several significant challenges during implementation:

- **MQTT Connection Errors:** Maintaining a stable connection between the ESP devices, the HiveMQ Cloud broker, and the Flask backend was a major difficulty. Connections frequently dropped, especially when a high volume of messages were sent simultaneously.
- **Library Integration Issues:** Using libraries like `paho.mqtt.client` on Flask and `PubSubClient` on ESP required a deep understanding of their functionality. Compatibility issues and configuration errors were common.
- **Hardware Limitations:** The ESP boards have limited memory and processing resources, making it challenging to run multiple tasks at once, such as reading sensors, sending data, and listening for control commands.
- **Interface Development:** Designing and developing user interfaces on multiple platforms (Web, MIT App Inventor, React Native) required knowledge of different technologies and how they interact with the backend.

### **Solutions**

To overcome these challenges, the following solutions were implemented:

- **For MQTT Connection Errors:**
  - **Automatic Reconnection:** The ESP code was enhanced with a `reconnectMQTT()` function to automatically reconnect whenever the connection is lost.
  - **Using MQTT over SSL/TLS:** The system was migrated to use `WiFiClientSecure` on the ESP and configured with SSL/TLS for MQTT on Flask to ensure a more secure and stable connection.
- **For Library Integration:**
  - **Leveraging Official Documentation:** Extensive time was spent studying the official documentation for `paho.mqtt.client` and `PubSubClient` to understand key callback functions (`on_message`, `on_connect`) and state management.
  - **Step-by-Step Testing:** Each component was integrated and tested independently to quickly identify the source of errors.
- **For Hardware Limitations:**
  - **Code Optimization:** The code was optimized to minimize unnecessary memory and processing usage. For example, instead of continuously sending data, devices publish sensor readings on a fixed 10-second cycle.
- **For Interface Development:**
  - **API-Centric Approach:** A robust set of RESTful APIs was built on Flask, allowing the user interfaces to focus solely on display and interaction logic without needing to handle complex backend processing. This approach simplified development and made it easy to extend to other platforms.

## **V. Conclusion & Future Work**

### **Conclusion**

The project successfully built an IoT-based smart home system, achieving its initial objectives. The system demonstrated the ability to:

- Establish bidirectional communication between ESP devices, the MQTT broker, and the Flask backend.
- Collect and display sensor data in real-time.
- Remotely control actuators via a web interface and mobile applications.
- Manage users and devices efficiently.

The success of this project proves that this model is highly feasible and can be expanded in the future.

### **Future Work**

To further enhance the system's functionality and performance, the following improvements could be considered:

- AI & Machine Learning Integration:
  - Use historical sensor data to predict environmental trends within the home.
  - Develop machine learning algorithms to automate device adjustments (e.g., turning on a fan when a certain temperature threshold is reached) without direct user commands.
- User Interface Enhancements:
  - Develop a complete React Native version with all the features of the web interface.
  - Integrate advanced features like push notifications to alert users about important events (e.g., motion detection when no one is home).
- Performance Optimization: Migrate to a more powerful database like PostgreSQL or MySQL to handle larger datasets and concurrent requests more efficiently.



**INTERNATIONAL UNIVERSITY  
SCHOOL OF INFORMATION TECHNOLOGY (IT)**

**GRADING GUIDELINE FOR FINAL REPORT**

Name of Student: Phạm Tuấn Đăng Khoa

ID: ITITIU22087

Subject: Interner of Things

| Nu<br>mb<br>er | Content                                                                                                         |        | Score | Comment |
|----------------|-----------------------------------------------------------------------------------------------------------------|--------|-------|---------|
| 1              | Format (max 9%)                                                                                                 |        |       |         |
|                | - Font type                                                                                                     | Yes No |       |         |
|                | - Font size                                                                                                     | Yes No |       |         |
|                | - Title                                                                                                         | Yes No |       |         |
|                | - Page number                                                                                                   | Yes No |       |         |
|                | - Table of contents                                                                                             | Yes No |       |         |
|                | - Header/Footer                                                                                                 | Yes No |       |         |
|                | - List of figures (if exists)                                                                                   | Yes No |       |         |
|                | - List of tables (if exists)                                                                                    | Yes No |       |         |
|                | - Report structure                                                                                              | Yes No |       |         |
| 2              | English Grammar and Spelling (max 6%)                                                                           |        |       |         |
|                | - Grammar                                                                                                       | Yes No |       |         |
|                | - Spelling                                                                                                      | Yes No |       |         |
| 3              | Data and Result Analysis (max 85%)<br>- Running result check<br>- Report contents<br>- Code with clear comments |        |       |         |
| Total Score    |                                                                                                                 |        |       |         |

Signature:

Date: